Collision-Avoiding Flocking With Multiple Fixed-Wing UAVs in Obstacle-Cluttered Environments: A Task-Specific Curriculum-Based MADRL Approach

Chao Yan, Chang Wang, Xiaojia Xiang, Kin Huat Low, Xiangke Wang, *Senior Member, IEEE*, Xin Xu, *Senior Member, IEEE*, and Lincheng Shen

***Abstract*— Multiple unmanned aerial vehicles (UAVs) are able to efficiently accomplish a variety of tasks in complex scenarios. However, developing a collision-avoiding flocking policy for multiple fixed-wing UAVs is still challenging, especially in obstacle-cluttered environments. In this article, we propose a novel curriculum-based multiagent deep reinforcement learning (MADRL) approach called task-specific curriculum-based MADRL (TSCAL) to learn the decentralized flocking with obstacle avoidance policy for multiple fixed-wing UAVs. The core idea is to decompose the collision-avoiding flocking task into multiple subtasks and progressively increase the number of subtasks to be solved in a staged manner. Meanwhile, TSCAL iteratively alternates between the procedures of online learning and offline transfer. For online learning, we propose a hierarchical recurrent attention multiagent actor–critic (HRAMA) algorithm to learn the policies for the corresponding subtask(s) in each learning stage. For offline transfer, we develop two transfer mechanisms, i.e., model reload and buffer reuse, to transfer knowledge between two neighboring stages. A series of numerical simulations demonstrate the significant advantages of TSCAL in terms of policy optimality, sample efficiency, and learning stability. Finally, the high-fidelity hardware-in-the-loop (HITL) simulation is conducted to verify the adaptability of TSCAL. A video about the numerical and HITL simulations is available at https://youtu.be/R9yLJNYRIqY.**

Manuscript received 5 July 2022; revised 8 December 2022; accepted 6 February 2023. Date of publication 23 February 2023; date of current version 6 August 2024. This work was supported in part by the Science and Technology Innovation 2030-Key Project of New Generation Artificial Intelligence under Grant 2020AAA0108200, in part by the National Natural Science Foundation of China under Grant 61825305 and Grant 61906203, and in part by the Postgraduate Scientific Research Innovation Project of Hunan Province under Grant CX20210001. The work of Chao Yan was supported by the China Scholarship Council. *(Corresponding author: Xiaojia Xiang.)*

Chao Yan is with College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, China, and also with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore 639798 (e-mail: yanchao17@nudt.edu.cn).

Chang Wang, Xiaojia Xiang, Xiangke Wang, Xin Xu, and Lincheng Shen are with the College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, China (e-mail: wangchang07@nudt.edu.cn; xiangxiaojia@nudt.edu.cn; xkwang@nudt.edu.cn; xinxu@nudt.edu.cn; lcshen@nudt.edu.cn).

Kin Huat Low is with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore 639798 (e-mail: mkhlow@ntu.edu.sg).

Digital Object Identifier 10.1109/TNNLS.2023.3245124

***Index Terms*— Curriculum learning, fixed-wing unmanned aerial vehicles (UAVs), flocking, multiagent deep reinforcement learning (MADRL), obstacle avoidance.**

I. Introduction

FLOCKING is an ubiquitous form of collective behavior in nature [1], [2]. Guided by only simple rules and limited environmental information, a group of individuals, such as a flock of birds and a school of fish, can achieve orderly motions [3]. Early in the 1980s, Reynolds [4] first simulated the flocking behavior through three basic rules, i.e., cohesion, alignment, and separation. Due to the utility of flocking models, it has potential applications in a variety of fields, including statistical physics, social dynamics, and swarm robotics [5]. Thus, the flocking problem has been widely investigated under various settings for many years [6], [7], [8].

In the robotics community, flocking with multiple unmanned aerial vehicles (UAVs) has attracted increasing attention owing to the characteristics of multiple UAVs, i.e., high maneuverability, strong robustness, and high survivability [9], [10], [11]. Different from rotary wings, fixed-wing UAVs have huge advantages in endurance, flight speed, and payload capacity [12]. Thus, flocking with fixed-wing UAVs has great potential for both civilian and military applications [13]. Nevertheless, the dynamics of fixed-wing UAVs are complex due to the nonholonomic constraints [14]. Consequently, developing a collision-avoiding flocking control approach for multiple fixed-wing UAVs is still challenging.

In recent years, many studies have been carried out to solve the flocking control problem by resorting to learning-based approaches, including deep learning [15], reinforcement learning [16], and imitation learning [17]. In particular, deep reinforcement learning (DRL), which combines the advantages of deep learning and reinforcement learning, has been proven effective across a wide range of tasks, such as path planning [18], collision/obstacle avoidance [19], [20], and autonomous driving [21], [22]. In view of this, an increasing number of researchers have applied DRL to the flocking control of multiple UAVs [23], [24], [25]. However, the existing

2162-237X © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

studies simplify the flocking control problem by assuming that UAVs are particle-based agents or by ignoring environmental obstacles. Due to the minimum speed constraints, fixed-wing UAVs are unable to hover at a fixed location, making the obstacle avoidance problem even more intractable. As far as we know, there is no published work on DRL-based flocking with multiple fixed-wing UAVs in obstacle-cluttered environments.

Employing existing DRL algorithms to directly learn the flocking and obstacle avoidance policy for multiple fixed-wing UAVs poses tremendous difficulties. First, the complexity and uncertainty of cluttered environments complicate the interactions between multiple UAVs. Learning a policy from scratch for such a complex task typically requires excessive training time. Besides, the sensing capacity of the UAVs is imperfect due to the limited field of view (FOV), which means the UAVs can only perceive partial information about the environment. As a result, the UAVs may fail to learn an effective collision-avoiding flocking policy in obstacle-cluttered environments.

More recently, the emergence of the curriculum learning paradigm [26] has provided new opportunities to handle the above difficulties. The main idea is to divide the learning process into a series of subtasks with growing difficulty levels. Leveraging this training technique, the learning speed and sample efficiency can be significantly improved [27], [28], [29]. However, most of the existing studies focus on single-agent tasks. How to properly generate a curriculum for multiagent tasks is still an open question.

In this article, we design a *task-specific curriculum* to learn the decentralized collision-avoiding flocking policy for multiple fixed-wing UAVs in obstacle-cluttered environments. Specifically, we decompose the final task into multiple subtasks and progressively increase the number of subtasks to be solved in each learning stage to schedule the curriculum. Then, we propose a multiagent actor–critic algorithm and two knowledge transfer mechanisms for *online learning* in each given stage and *offline transfer* across two neighboring stages, respectively. By alternating the online learning and offline transfer procedures, the proposed task-specific curriculum learning-based approach shows remarkable advantages over learning from scratch in terms of policy optimality, sample efficiency, and learning stability. To the best of our knowledge, this is the first curriculum learning enhanced multiagent DRL (MADRL) approach to solve the flocking and obstacle avoidance problem for a squad of fixed-wing UAVs. The main contributions of this article are summarized as follows.

1) The **t**ask-**s**pecific **c**urriculum-based M**A**DR**L** approach (TSCAL) is proposed to efficiently and steadily learn the flocking with obstacle avoidance policy for multiple fixed-wing UAVs by dividing the learning process into multiple stages and increasing the number of subtasks to be solved stage by stage.
2) A novel multiagent actor–critic reinforcement learning algorithm called hierarchical recurrent attention multiagent actor–critic (HRAMA) is proposed to efficiently learn the policy for the corresponding subtask(s) in each stage.
3) Two transfer mechanisms, i.e., model reload and buffer reuse, are developed to accelerate the learning process by transferring knowledge between two neighboring stages in an offline manner.

The rest of this article is organized as follows. Section II introduces the related work. Section III formulates the flocking with obstacle avoidance problem in the context of MADRL. Section IV proposes the curriculum-based MADRL approach. Section V presents the simulation results. Finally, Section VI provides the concluding remarks.

II. Related Work

A. Flocking With Multiple UAVs via DRL

DRL has been proven effective for the flocking control of rotary UAVs. For example, Shen et al. [23] proposed an improved deep deterministic policy gradient (DDPG) algorithm to achieve the flocking motion of a multiple UAV system with the help of digital twin technology. Khan et al. [24] proposed the graph policy gradient algorithm to learn the control policy parameterized by graph convolutional networks for large-scale quadrotor UAVs by leveraging the underlying graph symmetry among the UAVs. However, the above-mentioned methods assume the UAVs are particle-based agents, which is not applicable to fixed-wing UAVs [30].

Not many studies have been conducted on flocking with fixed-wing UAVs. Hung and Givigi [30] first applied a traditional RL algorithm to handle the flocking problem with fixed-wing UAVs in a stochastic environment. Following the settings of [30], we proposed a DRL-based approach to solve the flocking problem with fixed-wing UAVs in continuous state and action spaces [31]. More recently, in [32], we constructed local situation maps representing the environmental state to learn the collision-free flocking policy for a scalable squad of fixed-wing UAVs. In [33], we designed an attention-based embedding module to deal with the flocking and collision avoidance problem for a large-scale fixed-wing UAV swarm.

However, none of the above-mentioned studies took into account the environmental obstacles, assuming that the UAVs flew in free spaces. So far, there have been no published studies on flocking with obstacle avoidance of multiple fixed-wing UAVs in the context of DRL. In this article, we seek to learn a collision-avoiding flocking policy for multiple fixed-wing UAVs in an obstacle-cluttered environment.

B. DRL-Based Obstacle Avoidance for UAVs

Different from traditional methods that formulate obstacle avoidance as an optimization problem [34], [35], [36], the DRL-based approaches solve the problem in a reactive manner without any prior knowledge of the kinematic model. DRL has been successfully applied for obstacle avoidance of UAVs. Wang et al. [37] proposed the DRL with nonexpert helpers (LwH) algorithm to solve the UAV navigation and obstacle avoidance problem, assuming that the environmental information was obtained by range finders with a 360° FOV. Considering a more challenging setting where range finders had a limited FOV, Jin et al. [38] developed a hierarchical and interlaced DRL framework for cooperative navigation and

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

obstacle avoidance tasks. Similarly, Zhang et al. [39] proposed an improved twin-delayed DDPG algorithm for autonomous navigation and obstacle avoidance in a 3-D environment. However, these methods were only evaluated with a single-rotary UAV, the effectiveness of multiple fixed-wing UAVs has yet to be proved.

As for fixed-wing UAVs, Wu et al. [40] designed a DRL-based reactive maneuver control method to solve the obstacle avoidance problem. However, this study considered only a single-fixed-wing UAV. In contrast, we consider a more challenging problem where a squad of fixed-wing UAVs flocks in an obstacle-cluttered environment. This means that the collisions of the UAVs with each other, as well as the collisions between the UAVs and the environmental obstacles, have to be avoided at the same time.

C. Curriculum Learning in DRL

Many studies have focused on the integration of curriculum learning with single-agent DRL. For example, De Souza et al. [27] presented a curriculum-based DRL approach to learn a decentralized multiagent pursuit policy by starting from a large capture radius which was decreased gradually. Xiao et al. [28] designed a curriculum that gradually reduced the gap dimensions to efficiently train an end-to-end controller that enabled a quadrotor to safely traverse a narrow gap. Anzalone et al. [29] proposed the reinforced curriculum learning approach that adjusted the environmental settings to generate a curriculum to progressively optimize an autonomous driving policy. The above studies have shown the great potential of curriculum learning in accelerating the learning speed of DRL.

In multiagent settings, progressively increasing the number of agents is a popular manner for curriculum generation [41], [42], [43]. However, this requires a population-invariant network architecture to process the dynamic number of agents. Instead, this article incorporates curriculum learning into MADRL by decomposing the target task into multiple subtasks and gradually increasing the number of subtasks to be solved in each learning stage. We will detail this task-specific curriculum learning approach in Section IV.

III. System Model and Problem Formulation

In this article, we consider a flocking scenario where a squad of fixed-wing UAVs flies in a 2-D obstacle-cluttered space with the leader-follower topology. As shown in Fig. 1, the UAV flock consists of one leader-UAV (leader) and n follower-UAVs (followers). The leader is controlled by external commands given by the operator or the ground control station. We aim to develop a flocking and obstacle avoidance policy for the followers.

A. System Model

1) Kinematic Model: The state of a UAV can be represented by a tuple $\xi := (x, y, \psi, \phi, v)$, where (x, y) is the planar position, ψ is the heading angle, ϕ is the roll angle, and v is the airspeed. The kinematic model of the fixed-wing UAV can

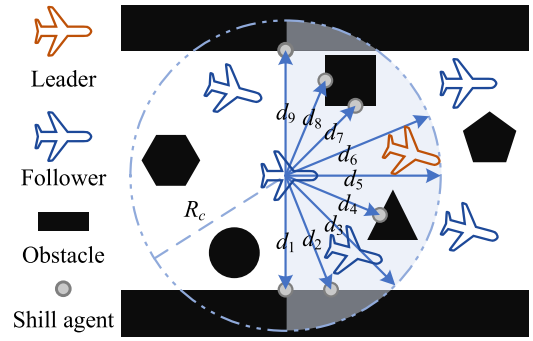


Fig. 1. Squad of fixed-wing UAVs consisting of one unique leader and n identical followers. $d_1, \ldots, d_9$ denote the distances between the follower and the surrounding obstacles in nine directions measured by distance sensors. The light blue area indicates the limited FOV of the UAV.

be formulated as follows:

$$\dot{\xi} = \frac{d}{dt}\begin{Bmatrix} x \\ y \\ \psi \\ \phi \\ v \end{Bmatrix} = \begin{Bmatrix} v\cos\psi + \eta_x \\ v\sin\psi + \eta_y \\ -(\alpha_g/v)\tan\phi + \eta_\psi \\ f_\phi(\phi, \phi_{\text{ref}}) \\ f_v(v, v_{\text{ref}}) \end{Bmatrix} \tag{1}$$

where α_g is the acceleration of gravity; and η_x, η_y, and η_ψ are the disturbance terms. The functions $f_\phi(\phi, \phi_{\text{ref}})$ and $f_v(v, v_{\text{ref}})$ describe the roll dynamics and airspeed dynamics [31], where the control inputs ϕ_{ref} and v_{ref} denote the desired roll angle and the desired airspeed of the UAV, respectively.

It is noted that the boundedness of the roll angle and the speed constraints must be considered to ensure the safety of UAVs. Besides, a fixed-wing UAV is constrained by the maximum acceleration and the maximum roll rate [14]. Mathematically, the constraints are formulated as follows:

$$\begin{cases} -\phi_{\max} \le \phi \le \phi_{\max} \\ 0 < v_{\min} \le v \le v_{\max} \\ -\omega_{\max} \le \omega \le \omega_{\max} \\ -u_{\max} \le u \le u_{\max} \end{cases} \tag{2}$$

where $\phi_{\max}$ is the maximum roll angle of the UAV; $v_{\max}$ and $v_{\min}$ are the maximum and minimum speeds; $\omega := \phi_{\text{ref}} - \phi$ and $u := v_{\text{ref}} - v$ are the roll rate and the acceleration; and $\omega_{\max}$ and $u_{\max}$ are the maximum roll rate and the maximum acceleration, respectively.

2) Sensor Model: In order to enable UAVs to avoid obstacles autonomously, we assume that the UAVs can sense partial information about the surrounding obstacles through its equipped distance sensor, e.g., LiDAR [38], [39]. In contrast to the existing state-of-the-art obstacle avoidance approaches [37], [44] that use the LiDAR with 360° FOV, we consider a more challenging setting. As illustrated in Fig. 1, each UAV only uses a nine-directional sparse distance sensor with a limited FOV of 180° to detect the surrounding obstacles. The light blue semicircle area indicates the limited FOV of the UAV, and its radius R_c is the maximum sensing range of the UAV.

We use a set of *shill agents* [9] to represent the environmental information. Denote the distances measured by the distance sensor of the follower i in nine directions as

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

$[d_{i,1}, d_{i,2}, \ldots, d_{i,9}]$. If the sensor does not detect any obstacles in the m direction, $d_{i,m} = R_c$. We define the set of shill agents of follower i as

$$\boldsymbol{\mathcal{E}}_i := \left\{ \left(p_{i,m}, q_{i,m}\right) \middle| d_{i,m} < R_c, m = 1, 2, \ldots, 9 \right\} \tag{3}$$

where $(p_{i,m}, q_{i,m})$ represents the corresponding shill agent's position calculated by

$$\begin{cases} p_{i,m} = x_i + d_{i,m} \cos\left(\psi_i + \dfrac{\pi}{8}(m-5)\right) \\ q_{i,m} = y_i + d_{i,m} \sin\left(\psi_i + \dfrac{\pi}{8}(m-5)\right) \end{cases} \tag{4}$$

where (x_i, y_i) and ψ_i are the follower i's position and heading angle, respectively.

B. Problem Formulation

We assume that the followers are identical and have the same limited sensing range R_c. The follower set is denoted as $\mathcal{I} := \{1, 2, \ldots, n\}$, where n is the number of the followers. The set of *neighbors* of the follower i at the time step t is defined as

$$\mathcal{N}_i^t = \left\{ j \middle| D_{i,j}^t < R_c, j \in \mathcal{I}, j \neq i \right\} \tag{5}$$

where $D_{i,j}^t$ is the distance from follower i to another follower j at the time step t. It is obvious that the interaction topology and the number of neighbors are time varying. We assume that each follower can obtain the leader's state, its neighbors' states, and the obstacle information within its FOV. The objective of the follower is to adjust its steering commands at each time step to minimize the average distance from the leader. Meanwhile, the collisions with the leader, the neighbors, as well as the surrounding obstacles have to be avoided. Thus, this problem can be formulated as follows:

$$\min \frac{1}{nT} \sum_{i=1}^{n} \sum_{t=1}^{T} \rho_i^t \tag{6a}$$

$$\text{s.t. } R_s < \rho_i^t < R_l \quad \forall i \in \mathcal{I} \quad \forall t \tag{6b}$$

$$D_{i,j}^t > R_s \quad \forall i \in \mathcal{I} \quad \forall j \in \mathcal{N}_i \quad \forall t \tag{6c}$$

$$D_{i,m}^t > R_s \quad \forall i \in \mathcal{I} \quad \forall m \quad \forall t \tag{6d}$$

where ρ_i^t is the distance from follower i to the leader at the time step t; R_s is the minimum safety radius; R_l is the maximum communication range of the leader; and $D_{i,m}^t$ is the distance from follower i to the mth obstacle at the time step t.

In the context of MADRL, problem (6) can be modeled as a decentralized partially observable Markov decision process (Dec-POMDP) [45].

1) Observation Space: For each follower $i \in \mathcal{I}$, its observation can be represented as

$$o_i := \left(\xi_l, \xi_i, \boldsymbol{\xi}_{\mathcal{N}_i}, \boldsymbol{\mathcal{E}}_i\right) \tag{7}$$

where $\xi_l := (x_l, y_l, \psi_l, \phi_l, v_l)$ is the leader's state, $\xi_i := (x_i, y_i, \psi_i, \phi_i, v_i)$ is the follower i's state, $\boldsymbol{\xi}_{\mathcal{N}_i} := \{\xi_j \mid j \in \mathcal{N}_i\}$ is the states of the follower i's neighbors, and $\boldsymbol{\mathcal{E}}_i$ denotes the positions of the shill agents that indicates the environmental information perceived by the follower i.

2) Action Space: The followers are maneuvered by adjusting their roll angle and speed setpoints. For each follower i, we define the action $a_i := (\omega_i, u_i)$ in continuous spaces, where $\omega_i \in [-\omega_{\max}, \omega_{\max}]$, $u_i \in [-u_{\max}, u_{\max}]$. Given the follower i's current roll angle ϕ_i, the next roll angle setpoint ϕ_{ref}^i is calculated by

$$\phi_{\text{ref}}^i = \begin{cases} \phi_{\max}, & \text{if } \phi_i + \omega_i > \phi_{\max} \\ -\phi_{\max}, & \text{if } \phi_i + \omega_i < -\phi_{\max} \\ \phi_i + \omega_i, & \text{otherwise.} \end{cases} \tag{8}$$

Similarly, the next speed setpoint v_{ref}^i is calculated by

$$v_{\text{ref}}^i = \begin{cases} v_{\max}, & \text{if } v_i + u_i > v_{\max} \\ v_{\min}, & \text{if } v_i + u_i < v_{\min} \\ v_i + u_i, & \text{otherwise} \end{cases} \tag{9}$$

where v_i is the follower i's current speed.

3) Reward Function: In the flocking with obstacle avoidance task, each follower aims to minimize the distance from the leader while avoiding collisions between the neighboring UAVs and the surrounding obstacles. Thus, we define the reward function for the follower i as follows:

$$r_i = r_i^{\text{fol}} + \sum_{j=1}^{|\mathcal{N}_i|} r_{i,j}^{\text{col}} + \sum_{m=1}^{|\boldsymbol{\mathcal{E}}_i|} r_{i,m}^{\text{obs}}. \tag{10}$$

The first term r_i^{fol} is specified to facilitate the follower to follow the leader, defined as

$$r_i^{\text{fol}} = \begin{cases} -P, & \rho_i \leq R_s \\ w_1\left(1 - e^{-\lambda_1(\rho_i - R_a)}\right), & R_s < \rho_i \leq R_a \\ w_2\left(1 - e^{-\lambda_2(R_a - \rho_i)}\right), & R_a < \rho_i \leq R_l \\ -P, & \rho_i > R_l \end{cases} \tag{11}$$

where R_a is the alert radius of the follower; $w_1 > 0$ and $w_2 > 0$ are two tuning parameters; λ_1 and λ_2 are two small positive constants; and P is a large positive constant.

The last two terms $r_{i,j}^{\text{col}}$ and $r_{i,m}^{\text{obs}}$ are designed to prevent the follower i from colliding with its jth neighbor and the mth shill agent, respectively. Specifically, $r_{i,j}^{\text{col}}$ is defined as

$$r_{i,j}^{\text{col}} = \begin{cases} -P, & D_{i,j} \leq R_s \\ w_1\left(1 - e^{-\lambda_1(D_{i,j} - R_a)}\right), & R_s < D_{i,j} \leq R_a \\ 0, & D_{i,j} > R_a. \end{cases} \tag{12}$$

Similarly, $r_{i,m}^{\text{obs}}$ is calculated by

$$r_{i,m}^{\text{obs}} = \begin{cases} -P, & d_{i,m} \leq R_s \\ w_1\left(1 - e^{-\lambda_1(d_{i,m} - R_a)}\right), & R_s < d_{i,m} \leq R_a \\ 0, & d_{i,m} > R_a. \end{cases} \tag{13}$$

IV. Curriculum-Based MADRL for Flocking With Obstacle Avoidance

This section starts by describing an overview of the TSCAL approach for flocking with obstacle avoidance of multiple fixed-wing UAVs. Then, we detail the two core procedures of TSCAL. Finally, we analyze the characteristics and time complexity of TSCAL.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

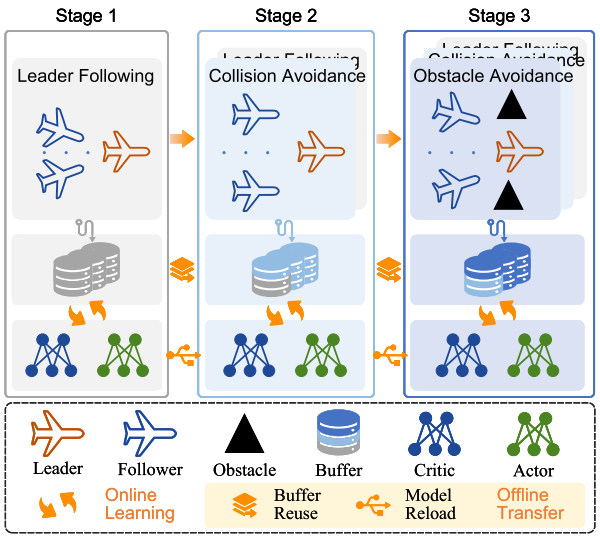


Fig. 2. Overview of the proposed TSCAL approach for collision-avoiding flocking with multiple fixed-wing UAVs in obstacle-cluttered environments.

A. Overview of TSCAL

The performance of MADRL algorithms is typically influenced by the complexity of tasks. Due to the complexity of the environment and the dynamic interactions between the UAVs, it is difficult for multiple fixed-wing UAVs to directly learn a collision-avoiding flocking policy in obstacle-cluttered environments using existing MADRL algorithms. Inspired by the human learning process that starts with easy examples of a task and then gradually increases the task difficulty, one promising approach is to first learn the policy for a similar but simpler source task and then gradually transfer to a target task. The learned knowledge in the source task will accelerate the learning process of the target task. For this purpose, we design a *task-specific curriculum* for MADRL in the curriculum learning paradigm.

An overview of the proposed TSCAL approach is summarized in Fig. 2. The core insight is to divide the learning process into three learning stages and gradually increase the task difficulty in each stage. Unlike the existing curriculum learning methods that generate a task sequence by adjusting the environmental settings [27], [28], [29] or increasing the number of agents progressively [41], [42], [43], we evaluate the task difficulty with the number of subtasks to be solved. To this end, inspired by the bio-flocking models [4], [6] as well as the behavior-based formation control methods [46], [47], we decompose the collision-avoiding flocking task into three subtasks.

1) *Leader Following:* This follows the leader with an appropriate distance.
2) *Collision Avoidance:* This avoids conflicts between the nearby followers.
3) *Obstacle Avoidance:* This avoids collisions with the environmental obstacles.

TSCAL consists of two core procedures: the *online learning* in a given stage and the *offline transfer* across two successive stages. During the online learning procedure, the agents (i.e., the followers) learn the policies for the corresponding subtask(s) given in the current stage using the experience samples collected through online interactions with the environments. During the offline transfer procedure, we transfer the knowledge learned from the previous stage to the next stage in an offline manner by reloading the learned models and reusing the collected experiences. Specifically, in stage 1, we start with a squad of fixed-wing UAVs to learn a policy for the leader following subtask from scratch. In stage 2, we first reload the model learned from stage 1 as the initialization, and then reuse the previous experience replay buffer to pretrain in an offline manner. Afterward, we optimize the same policy for the leader following and collision avoidance subtasks with online experience samples. Similarly, in stage 3, we further consider the obstacle avoidance subtask and repeat the two procedures to obtain the final policy. The details of the online learning and offline transfer procedures are described in Sections IV-B and IV-C, respectively.

B. Online Learning

In this section, we propose a novel multiagent actor–critic reinforcement learning algorithm on top of the multiagent DDPG (MADDPG) [48] algorithm and the hierarchical attention mechanism [49] for online learning. We refer to the proposed algorithm as HRAMA.

In contrast to the vanilla MADDPG, HRAMA has three major improvements. First, the parameter sharing [50] technique is introduced. Considering the followers in our flocking scenario are identical, we allow the different followers to share a flocking policy with the same parameters. This technique significantly reduces the number of parameters and improves sample efficiency. Second, a memory component implemented by recurrent neural networks is integrated to deal with the partially observable problem [51]. Considering the structure of gated recurrent units (GRU) [52] is simpler, with fewer parameters and faster training than long short-term memory networks (LSTM), we use GRU to encode the historical information and then store it into the experience replay buffer. Third, in order to handle the dynamic-dimensional input and discriminatively select relevant information for each agent, a novel hierarchical attention network (HAN) is designed by leveraging the hierarchical attention mechanism [49]. We will detail its architecture in Section IV-B2.

1) Learning Process: We perform the following updates for the optimization of HRAMA. Due to parameter sharing, HRAMA only needs to learn a shared actor and a centralized critic for all agents. The actor and the critic are allowed to optimize the experiences of all agents simultaneously. Formally, denote the actor and the critic as μ and Q, respectively. Given a mini-batch of experiences sampled from the experience replay buffer $\mathcal{D}$, the actor μ parameterized by θ^{μ} is optimized with the policy's gradient

$$\nabla_{\theta^{\mu}} \mathcal{J}(\theta^{\mu}) = \mathbb{E}_{\mathbb{U}(\mathcal{D})}\left[\frac{1}{n}\sum_{i=1}^{n}\left(\nabla_{\theta^{\mu}}\mu\left(o_i^t, h_i^{t-1}, a_i^{t-1}\right) \times \nabla_{a_i} Q\left(\boldsymbol{o}^t, \boldsymbol{h}^{t-1}, a_1^t, \ldots, a_n^t\right)\Big|_{a_i^t=\mu\left(o_i^t, h_i^{t-1}, a_i^{t-1}\right)}\right)\right] \tag{14}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

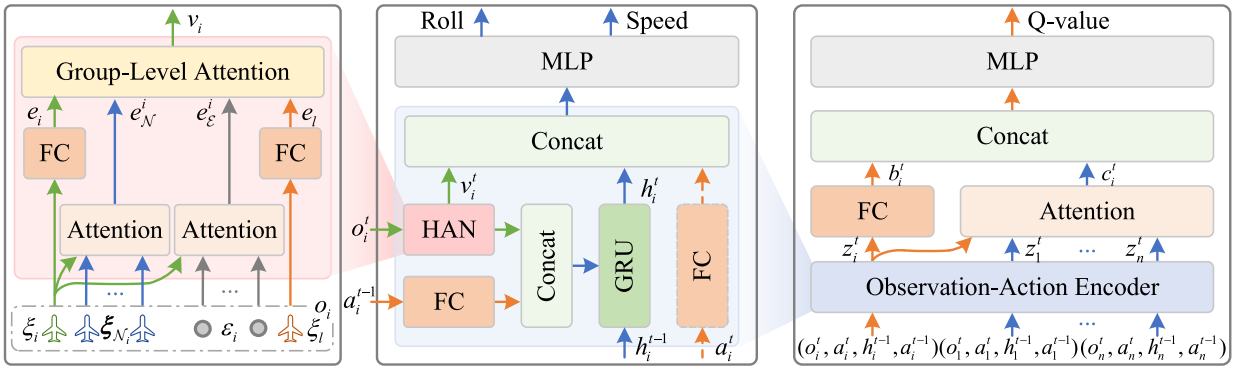


Fig. 3. Network structure of HRAMA. (a) HAN module. (b) Actor-network structure. Note that the layer marked by the dotted box is only used for the critic. (c) Critic network structure.

where o_i^t and a_i^t are the observation and action of agent i at the time step t. h_i^{t-1} and a_i^{t-1} are the last action and historical information of agent i at the time step $t-1$, respectively. Note $\boldsymbol{o}^t := (o_1^t, \ldots, o_n^t)$, $\boldsymbol{h}^{t-1} := (h_1^{t-1}, \ldots, h_n^{t-1})$.

The critic's parameter θ_i^Q is updated by minimizing the loss

$$\mathcal{L}(\theta^Q) = \mathbb{E}_{\mathbb{U}(\mathcal{D})}\left[\frac{1}{n}\sum_{i=1}^{n}\left(Q\left(\boldsymbol{o}^t, \boldsymbol{h}^{t-1}, \boldsymbol{a}^{t-1}, \boldsymbol{a}^t\right) - y_i^t\right)^2\right] \tag{15}$$

where $\boldsymbol{a}^t := (a_1^t, \ldots, a_n^t)$ is the concatenation of all agents' actions at the time step t. The target Q value y_i^t is defined as

$$y_i^t = r_i^t + \gamma\, Q'\left(\boldsymbol{o}^{t+1}, \boldsymbol{h}^t, \boldsymbol{a}^t, a_1^{t+1}, \ldots, a_n^{t+1}\right)\Big|_{a_j^{t+1}=\mu'\left(o_j^{t+1}, h_j^t, a_j^t\right)} \tag{16}$$

where r_i^t is the reward received by agent i at the time step t, $\gamma \in [0, 1)$ is the discounted factor. μ' and Q' denote the target networks. Their parameters $\theta^{\mu'}$ and $\theta^{Q'}$ are updated as

$$\begin{cases} \theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'} \\ \theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'} \end{cases} \tag{17}$$

where $\tau \ll 1$ is the soft update rate.

2) Network Architecture: Fig. 3 depicts the network structure of HRAMA. We first describe the actor network and then the critic network.

a) Actor network: As shown in Fig. 3(b), the action of each follower i is determined as follows:

$$\mu\left(o_i^t, h_i^{t-1}, a_i^{t-1}\right) = \mathrm{MLP}\left(v_i^t \,||\, h_i^t\right) \tag{18}$$

where

$$v_i^t = \mathrm{HAN}\left(o_i^t\right),\ h_i^t = \mathrm{GRU}\left(v_i^t \,||\, \mathrm{FC}\left(a_i^{t-1}\right), h_i^{t-1}\right). \tag{19}$$

Here MLP$(\cdot)$ is a three-layer multilayer perceptron (MLP). $||$ is the concatenation operation. We will explain v_i^t and HAN$(\cdot)$ in the next paragraph. GRU$(\cdot)$ is a GRU layer that records the information of historical observations and actions. FC$(\cdot)$ is a fully connected (FC) layer.

We now define the structure of HAN. As shown in Fig. 3(a), the designed HAN module is used to encode the variable-length observation o_i into the fixed-length embedding v_i of agent i. Considering that o_i consists of four groups, i.e., $(\xi_l, \xi_i, \boldsymbol{\xi}_{\mathcal{N}_i}, \boldsymbol{\mathcal{E}}_i)$, we first use four embedding functions to encode them separately. As a result, the embeddings of the leader's state ξ_l and the follower i's state ξ_i can be easily represented as

$$e_l = \mathrm{FC}(\xi_l),\ e_i = \mathrm{FC}(\xi_i). \tag{20}$$

Nevertheless, since the numbers of neighbor followers and shill agents are uncertain, we customize an attention embedding block [53] to separately aggregate the states of the neighbors $\boldsymbol{\xi}_{\mathcal{N}_i}$ and the states of the shill agents $\boldsymbol{\mathcal{E}}_i$. The aggregated embedding of $\boldsymbol{\xi}_{\mathcal{N}_i}$ is computed by

$$e_{\mathcal{N}}^i = \mathrm{Attention}\left(\boldsymbol{\xi}_{\mathcal{N}_i}, \xi_i\right) = \sum_{j\in\mathcal{N}_i} w_{\mathcal{N}}^{i,j}\mathrm{FC}\left(\xi_j\right). \tag{21}$$

The attention weight $w_{\mathcal{N}}^{i,j}$ is calculated as

$$w_{\mathcal{N}}^{i,j} = \frac{\exp\left(q_{\mathcal{N}}^{i,j}\right)}{\sum_j \exp\left(q_{\mathcal{N}}^{i,j}\right)},\quad q_{\mathcal{N}}^{i,j} = \frac{\xi_i^{\mathrm{T}} W_{\mathcal{N}}^{\mathrm{T}}\mathrm{FC}\left(\xi_j\right)}{\sqrt{d_k}} \tag{22}$$

where $W_{\mathcal{N}}$ denotes the parameter to learn and d_k is the dimension of FC(ξ_j). Likewise, we can derive the aggregated embedding of $\boldsymbol{\mathcal{E}}_i$ by

$$e_{\mathcal{E}}^i = \mathrm{Attention}(\boldsymbol{\mathcal{E}}_i, \xi_i). \tag{23}$$

Then, we design a group-level attention block to aggregate the embeddings of four groups. In contrast to directly concatenating them, we compute the attention embedding v_i by a weighted sum of each group's embedding

$$v_i = \sum_g w_{\mathcal{G}}^{i,g}\mathrm{FC}\left(e_g\right),\quad e_g \in \left\{e_l, e_i, e_{\mathcal{N}}^i,\ e_{\mathcal{E}}^i\right\}. \tag{24}$$

The group-level attention weight $w_{\mathcal{G}}^{i,g}$ is defined as

$$w_{\mathcal{G}}^{i,g} = \frac{\exp\left(q_{\mathcal{G}}^{i,g}\right)}{\sum_g\left(\exp\left(q_{\mathcal{G}}^{i,g}\right)\right)},\quad q_{\mathcal{G}}^{i,g} = \mathrm{MLP}\left(e_g\right). \tag{25}$$

We note that the designed group-level attention block is invariant to the number of groups. This property will contribute to the knowledge transfer between different learning stages (see Section IV-C1 for more details).

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

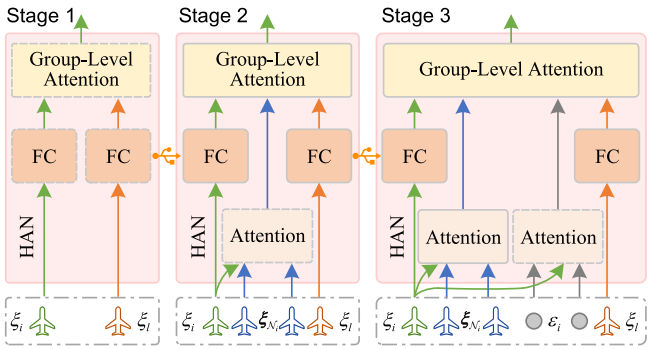


Fig. 4. Model reload with the HAN architecture. The dashed boxes denote the network layers/modules with randomly initialized weights. The solid boxes represent the layers/modules initialized with the parameters from the previous stage.

b) Critic network: As shown in Fig. 3(c), we estimate each follower i's Q function as follows:

$$Q\left(\boldsymbol{o}^t, \boldsymbol{h}^{t-1}, \boldsymbol{a}^{t-1}, \boldsymbol{a}^t\right) = \mathrm{MLP}\left(b_i^t \,\|\, c_i^t\right) \tag{26}$$

where

$$b_i^t = \mathrm{FC}\left(z_i^t\right), c_i^t = \mathrm{Attention}\left(\left[z_1^t, \ldots, z_n^t\right], z_i^t\right). \tag{27}$$

Here, z_i^t is the agent embedding of agent i, which is the output of an observation-action encoder. This encoder has a similar structure to the actor network. The only difference between them is that the input of the observation-action encoder contains the action a_i^t. Note that all agents share the same observation-action encoder, but the actor network and the critic network have different parameters.

C. Offline Transfer

In this section, we develop two offline transfer mechanisms, i.e., *model reload* and *buffer reuse*, to transfer knowledge between two neighboring stages.

1) Model Reload: Reloading the previously learned model as the initialization for a similar task can significantly improve sample efficiency. Thus, we present the first knowledge transfer mechanism called model reload. Specifically, in a new learning stage of our flocking task, we reload the parameters of the actor and the critic from the previous stage, instead of initializing them randomly.

It should be noted that the dimensions of observation are different in different stages. The variable-length input makes it infeasible to directly reload the previous model with the regular network architecture. Fortunately, the designed HAN architecture based on the hierarchical attention mechanism (see Section IV-B2) is invariant to the number and type of the entities. Using this architecture, the model learned from the previous stage is able to be reloaded and fine-tuned in the current learning stage.

As illustrated in Fig. 4, we gradually increase the network branches/modules in different stages to learn the control policies for more subtasks. In stage 1, we learn the policy for the leader following subtasks only. Thus, the input only contains the states of the leader and the follower itself, rather than the overall observation. We randomly initialize the parameters for the actor and the critic in this stage. In stage 2, we further consider the states of the neighbor followers for the leader following and collision avoidance subtasks. We reload the model learned from stage 1 and add an attention module with randomly generated weights to encode the neighbors' states. Similarly, in stage 3, taking the full observation as input, we further optimize the previous model to obtain the final policy.

2) Buffer Reuse: The second knowledge transfer mechanism is called buffer reuse. In each stage, the followers are equipped with an experience replay buffer to learn the policies for different subtasks. Thus, apart from reloading the learned model from the previous stage, we can reuse the previously collected experiences to accelerate the learning process.

It should be noted that the learning objective, i.e., the reward function, is different in different stages. In stage 1, we learn a policy for the leader following subtask with the learning following reward (11) only. In stage 2, we further consider the collision avoidance reward (12) to update the policy for the leader following and collision avoidance subtasks. Similarly, in stage 3, we use the overall reward, including the obstacle avoidance reward (13), to learn the final policy. Formally, the multistage reward function is reshaped as

$$r_{i,k} = \begin{cases} r_i^{\mathrm{fol}}, & k = 1 \\ r_i^{\mathrm{fol}} + \sum_{j=1}^{|\mathcal{N}_i|} r_{i,j}^{\mathrm{col}}, & k = 2 \\ r_i^{\mathrm{fol}} + \sum_{j=1}^{|\mathcal{N}_i|} r_{i,j}^{\mathrm{col}} + \sum_{m=1}^{|\boldsymbol{\varepsilon}_i|} r_{i,m}^{\mathrm{obs}}, & k = 3. \end{cases} \tag{28}$$

Let $\mathcal{D}^k$ be the experience replay buffer that stores the corresponding experience samples for each stage $k \in \{1, 2, 3\}$. At the arrival of a new learning stage k, we first restore the experience replay buffer $\mathcal{D}^{k-1}$ from the previous stage $k-1$. Then, according to the multistage reward function (28), we recalculate the rewards of the experience samples to match the learning objective of the stage k

$$\mathcal{D}^k = \left\{\left(\boldsymbol{o}, \boldsymbol{a}, \boldsymbol{r}_k, \boldsymbol{o}', \ldots\right) \,\middle|\, \left(\boldsymbol{o}, \boldsymbol{a}, \boldsymbol{r}, \boldsymbol{o}', \ldots\right) \in \mathcal{D}^{k-1}\right\} \tag{29}$$

where $\boldsymbol{r}_k := (r_{1,k}, \ldots, r_{n,k})$ are concatenation of all agents' reward for the stage k. Afterward, we sample batches of experiences from $\mathcal{D}^k$ and update the actor and the critic in an offline manner with (14) and (15), respectively.

D. Summary and Analysis

We instantiate TSCAL by the proposed HRAMA reinforcement learning algorithm (see Section IV-B). The overall workflow is summarized in Algorithm 1. In consistency with MADDPG [48], our algorithm follows the centralized-training and decentralized-execution (CTDE) paradigm. During the training phase, the experiences of all the followers are stored in a shared experience replay buffer, and the shared policy and critic are optimized with the experiences sampled from the buffer randomly. During the execution phase, each follower independently determines and executes their action according to their unique observations.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 TSCAL With HRAMA

Input: N − experience replay buffer maximum capacity;
N_p − pre-training time steps;
N_s − maximum time step per episode;
N_b − training batch size;
C − update frequency of target networks

Initialize actor network parameters θ_0^{μ} and critic network parameters θ_0^{Q} randomly
Initialize experience replay buffer $\mathcal{D}^0$ with capacity N
for stage $k = 1$ to 3 **do**
  /**Offline Transfer**/
  Reload models from the previous stage:
  $\theta_k^{\mu} \leftarrow \theta_{k-1}^{\mu}$; $\theta_k^{Q} \leftarrow \theta_{k-1}^{Q}$;
  $\theta_k^{\mu'} \leftarrow \theta_{k-1}^{\mu}$; $\theta_k^{Q'} \leftarrow \theta_{k-1}^{Q}$ ▷ Model Reload
  Restore experience buffer from previous stage:
  $\mathcal{D}^k \leftarrow \mathcal{D}^{k-1}$ ▷ Buffer Reuse
  Relabel the rewards with Eq. (29)
  for $t = 1$ to N_p **do** ▷ Pre-training
    Sample a minibatch of N_b tuples from $\mathcal{D}^k$
    Update θ_k^{μ} using the policy's gradient Eq. (14)
    Update θ_k^{Q} by minimizing the loss of Eq. (15)
    Update target network parameters $\theta_k^{\mu'}$ and $\theta_k^{Q'}$ every C time steps with Eq. (17)
  end for
  /**Online Learning**/
  Reset exploration parameter σ
  repeat (for each episode)
    Initialize the environmental state randomly
    Initialize last action $\boldsymbol{a}^0 \leftarrow \mathbf{0}$
    Initialize historical information $\boldsymbol{h}^0 \leftarrow \mathbf{0}$
    Receive initial observations $\boldsymbol{o}^1$
    for $t = 1$ to N_s **do**
      Select action with current policy and Gaussian noise: $a_i^t = \mu(o_i^t, h_i^{t-1}, a_i^{t-1}) + \mathbb{N}(0, \sigma^2)$
      Update historical information h_i^t with GRU
      Execute actions and update system states
      Calculate immediate reward r_i^t with Eq. (28)
      Receive new observations $\boldsymbol{o}^{t+1}$
      Store $(\boldsymbol{o}^t, \boldsymbol{h}^{t-1}, \boldsymbol{a}^{t-1}, \boldsymbol{a}^t, \boldsymbol{r}^t, \boldsymbol{o}^{t+1}, \boldsymbol{h}^t)$ into experience replay buffer $\mathcal{D}^k$
      $\boldsymbol{o}^t \leftarrow \boldsymbol{o}^{t+1}$; $\boldsymbol{h}^{t-1} \leftarrow \boldsymbol{h}^t$; $\boldsymbol{a}^{t-1} \leftarrow \boldsymbol{a}^t$
      Sample a minibatch of N_b tuples from $\mathcal{D}^k$
      Update θ_k^{μ} using the policy's gradient Eq. (14)
      Update θ_k^{Q} by minimizing the loss of Eq. (15)
      Update target network parameters $\theta_k^{\mu'}$ and $\theta_k^{Q'}$ every C time steps with Eq. (17)
    end for
  until desired number of episodes
end for

It is worth emphasizing that the proposed TSCAL approach is a *general framework* that is adaptive to various off-policy multiagent reinforcement learning algorithms. Our TSCAL method is able to handle various types of entities or agents. With the help of the designed HAN module, the interactive relationships between agents and between groups can be modeled explicitly and effectively. In the computation of agent-level attention, the time complexity for each agent is $O(N_g)$, where N_g represents the number of entities in a group. The time complexity of group-level attention is calculated as $O(K_g)$, where K_g denotes the number of groups. Since the agent-level attention and group-level attention blocks are cascaded, the final time complexity of making a decision for each agent i is $O(|\mathcal{N}_i|+|\boldsymbol{\mathcal{E}}_i|)$, where $|\mathcal{N}_i|$ and $|\boldsymbol{\mathcal{E}}_i|$ indicate the numbers of neighbors and shill agents for agent i, respectively. It is obvious that the time complexity is linearly related to the number of entities in the agent's observation.

V. Experiments and Results

In this section, we first introduce the experimental setup. Then, we describe the numerical simulation and the hardware-in-the-loop (HITL) simulation for the evaluation of the proposed approach.

A. Experimental Setup

1) Implementation Details: In our flocking scenario, we assumed that the obstacles in the workspace were regular polygons (including circles). The sizes and shapes were randomly generated from the predefined sets. Specifically, the size set was defined as {10, 15, 20, 25, 30, 35, 40}, which meant the radius of the regular polygon (in meters). The shape set was defined as {3, 4, 5, 6, $+\infty$}, which indicated the number of edges of the regular polygon. Note that $+\infty$ meant the shape of the obstacle was circular. We considered that the collision would occur if the distance between two followers was less than 5 m, i.e., the minimum safety distance $R_s = 5$ m. The maximum sensing range R_c and the alert radius of the followers R_a were set to 100 and 50 m, respectively.

In the HAN module, the FC layer in (24) had 256 neurons; the MLP for group-level attention (25) had 256, 256, and one neuron(s), and the last layer was activated by a softmax function. In the actor network, the GRU had 256 hidden units; the MLP had 256, 256, and two neurons, and the last layer was activated by a hyperbolic tangent (tanh) function. In the critic network, the FC layer in (27) had 256 neurons; the MLP had 256, 256, and one neuron(s), and the last layer was activated by a linear function. Each of the other FC layers in Fig. 3 had 64 neurons. We note that the default activation function for all layers was rectified linear unit (ReLU).

We used the PyTorch library in Python to implement our approach. All network parameters were optimized using the Adam optimizer with learning rates $\alpha = 0.001$ (for the actor) and $\beta = 0.0001$ (for the critic). We trained the proposed algorithm for 10 000 episodes with five random seeds. During the training, the Gaussian exploration parameter σ was annealed exponentially from 1.0 to 0.05. In each episode, the initial environmental state was generated randomly. Table I provides the empirical values for the parameter settings.

2) Performance Metrics: We defined the following three metrics to quantitatively evaluate our approach.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETER SETTINGS

Name	Value	Name	Value
R_l	250 m	R_c	100 m
R_a	50 m	R_s	5 m
α_g	9.8 m/s^2	$\phi_{\max}$	30°
$v_{\max}$	18 m/s	$v_{\min}$	12 m/s
$\omega_{\max}$	10°	$u_{\max}$	1 m/s
w_1	10	w_2	100
λ_1	0.1	λ_2	0.01
P	1000	N_p	3000
N_b	32	N_s	60
N	100000	γ	0.95
τ	0.01	C	10

1) *Average Reward (G):* The average reward for all followers was calculated as

$$G = \frac{1}{nT}\sum_{i=1}^{n}\sum_{t=1}^{T} r_i^t. \tag{30}$$

2) *Average Distance ($\bar{\rho}$):* The average distance between the leader and all the followers was computed by

$$\bar{\rho} = \frac{1}{nT}\sum_{i=1}^{n}\sum_{t=1}^{T} \rho_i^t. \tag{31}$$

3) *Collision Rate (F):* It was considered that a collision had occurred if the distance between two given UAVs was less than the minimum safety distance. Accordingly, the collision rate was defined as

$$F = \frac{1}{nT}\sum_{i=1}^{n}\sum_{t=1}^{T} \mathbb{1}\left(\min\left\{\rho_i^t, D_{i,\min}^{\text{nei}}, D_{i,\min}^{\text{obs}}\right\} < R_s\right) \tag{32}$$

where $\mathbb{1}(\cdot)$ denoted the indicator function. $D_{i,\min}^{\text{nei}} = \min\{D_{i,j}^t | j \in \mathcal{N}_i\}$ was the minimum distance from follower i to its neighbors. $D_{i,\min}^{\text{obs}} = \min\{D_{i,m}^t | m = 1, 2, \ldots\}$ was the minimum distance from follower i to the surrounding obstacles.

B. Simulation Results for HRAMA

We first compared our HRAMA algorithm with several benchmarks in a simple scenario.

1) Training Results: We selected the following four algorithms as the baselines, i.e., MADDPG [48], R-MADDPG [51], MAAC [54], and HAMA [49]. For a fair comparison, all four baselines shared parameters among the followers. Besides, MAAC and HAMA adopted the same base optimization algorithm as our HRAMA. We used the five algorithms separately to learn the final collision-avoiding flocking policies directly in a simple scenario with ten followers. Specifically, ten obstacles were randomly distributed in an area of 1200 m × 200 m. We assumed that the leader flew along the positive direction of the x-axis with a fixed speed $v_l = 15$ m/s.

Fig. 5 depicts the learning curves of the five algorithms. As can be seen, the five average reward curves rise at a similar rate in the early phase. This result means the

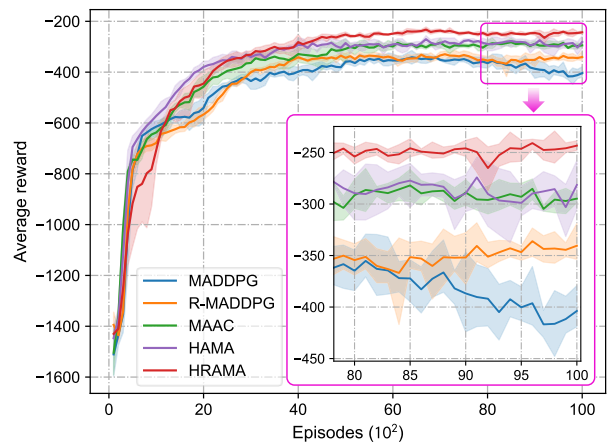


Fig. 5. Learning curves of MADDPG [48], R-MADDPG [51], MAAC [54], HAMA [49], and HRAMA (ours).

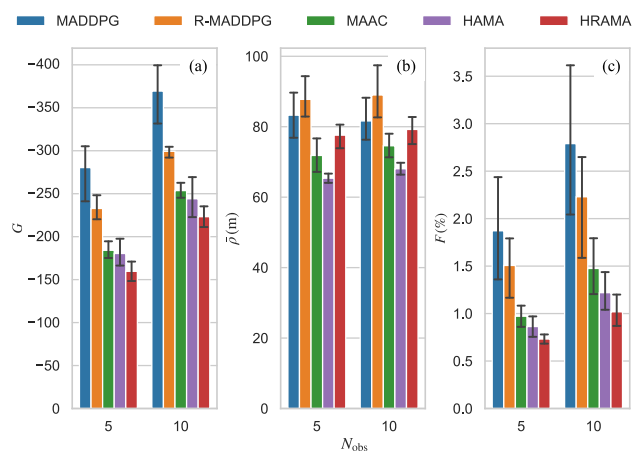


Fig. 6. Bar chart of the performance of the five learned flocking policies. Note that the higher the bar, the worse the performance. (a) Average reward (note that the vertical axis is inversed). (b) Average distance. (c) Collision rate.

five algorithms have similar learning efficiency. Nevertheless, MADDPG obtains the lowest average reward. Compared with MADDPG, the other three baselines, i.e., R-MADDPG, MAAC, and HAMA receive higher average rewards. These results validate that the memory component that encodes historical information and the attention mechanism in the critic are useful for improving performance. Overall, the proposed HRAMA algorithm shows superior performance in terms of the final reward. These comparison results demonstrate that the proposed HRAMA algorithm is superior to the four existing state-of-the-art MADRL algorithms.

2) Testing Results: After training, we evaluated the flocking policies learned with the five algorithms in the scenario with five and ten obstacles, respectively. We kept the environmental setting the same as during training. We tested with five random seeds for each policy. Each test consisted of 20 episodes and each episode lasted for 60-time steps.

Fig. 6 and Table II report the testing results of the five learned policies, where N_{obs} denotes the number of obstacles in the testing environment. As shown, MADDPG shows the worst performance in terms of average reward, average distance,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TESTING RESULTS OF THE LEARNED FLOCKING POLICIES USING DIFFERENT ALGORITHMS

N_{obs}	Algorithm	G	$\bar{\rho}\,(\text{m})$	$F\,(\%)$
5	MADDPG [48]	-280.40 $\pm$ 39.63	83.27 $\pm$ 7.56	1.87 $\pm$ 0.60
	R-MADDPG [51]	-232.82 $\pm$ 15.63	87.77 $\pm$ 6.75	1.51 $\pm$ 0.37
	MAAC [54]	-184.12 $\pm$ **11.57**	71.84 $\pm$ 5.46	0.97 $\pm$ 0.14
	HAMA [49]	-180.41 $\pm$ 17.09	**65.37** $\pm$ **1.55**	0.86 $\pm$ 0.13
	HRAMA (ours)	**-159.65** $\pm$ 12.37	77.60 $\pm$ 3.95	**0.73** $\pm$ **0.05**
10	MADDPG [48]	-369.41 $\pm$ 38.54	81.64 $\pm$ 7.15	2.79 $\pm$ 0.90
	R-MADDPG [51]	-299.35 $\pm$ **7.47**	89.01 $\pm$ 8.47	2.23 $\pm$ 0.61
	MAAC [54]	-253.61 $\pm$ 10.63	74.59 $\pm$ 3.81	1.48 $\pm$ 0.33
	HAMA [49]	-244.10 $\pm$ 27.94	**68.06** $\pm$ **1.99**	1.22 $\pm$ 0.24
	HRAMA (ours)	**-223.24** $\pm$ 13.94	79.22 $\pm$ 4.35	**1.02** $\pm$ **0.21**

and collision rate. Compared with MADDPG, the collision rate of R-MADDPG is much lower. The main reason is that the GRU-encoded historical information can alleviate the effects of partial observability. Whereas, the average distance of R-MADDPG is higher than that of MADDPG. This result indicates that the average distance and the collision rate are two competing metrics. Intuitively, the followers flying closer to the leader may lead to a higher collision probability. We can find the same conclusion by comparing HAMA with HRAMA. Although the average distance of HRAMA is higher than that of MAAC and HAMA, the collision rate and average reward are much better. In the case of $N_{\text{obs}} = 5$, the collision rate of HRAMA decreases by 60.96% compared with that of MADDPG. Even when compared with HAMA, the collision rate of HRAMA is reduced by 15.12%. This result shows that HRAMA has a significant advantage in terms of collision rate. To sum up, the proposed HRAMA algorithm outperforms the four baselines in safety and achieves the best tradeoff between the competing objectives. We can draw the same conclusion in the case of $N_{\text{obs}} = 10$. This means that our algorithm still performs better when the number of obstacles increases.

C. Simulation Results for TSCAL

We then evaluated the effectiveness of the proposed TSCAL approach in a complex scenario.

1) Training Results: In the training scenario, eighty obstacles were randomly distributed in an area of 1200 m $\times$ 1200 m. In each episode, the initial states of the leader and the followers were initialized randomly. The leader's predefined paths were various. At each time step, the leader's actions were determined randomly from the action space.

We used the proposed three-stage curriculum learning methods to learn the collision-avoiding policy with ten followers. In each stage, the policy was trained for 1000, 2000, and 7000 episodes. Between two neighboring stages, we used the model reload mechanism and the buffer reuse mechanism separately or jointly to transfer knowledge. As a result, we obtained three methods, i.e., *model reload (LCO)*, *buffer reuse (LCO)*, and *TSCAL (LCO)*, where *LCO* refers to the *leader following-collision avoidance-obstacle avoidance* subtask sequence (as aforementioned in Section IV-A). To investigate the effect of the order of subtasks, we changed the subtask sequence to *leader following-obstacle avoidance-collision avoidance (LOC)*, yielding the *TSCAL (LOC)* approach. For comparison, we directly trained a collision-avoiding flocking policy from scratch in an obstacle-cluttered environment as a baseline. We call this approach *learning from scratch*.

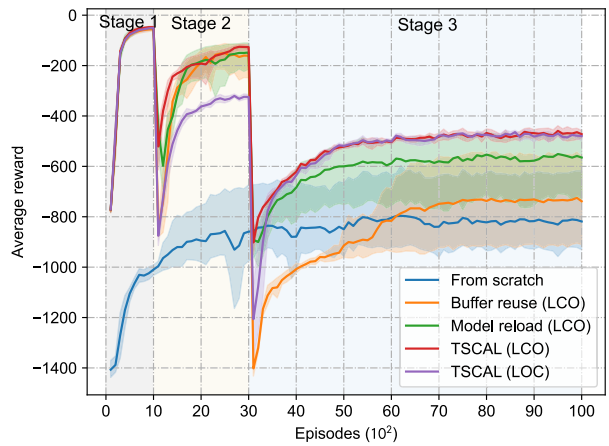


Fig. 7. Learning curves of curriculum learning approaches and learning from scratch.

Fig. 7 records the average rewards obtained by curriculum learning approaches and learning from scratch. It can be found that learning from scratch fails to obtain a stable policy. Compared with learning from scratch, the four multistage curriculum learning approaches, i.e., model reload (LCO), buffer reuse (LCO), TSCAL (LCO), and TSCAL (LOC) perform much better. This result suggests that it is important to divide the complex task into several easier subtasks. The result of model reload (LCO) is much better than that of buffer reuse (LCO). This means that the model reloads mechanism is more useful than the buffer reuse mechanism. By comparing TSCAL (LCO) and model reload (LCO), we can conclude that the buffer reuse mechanism can improve the learning speed in the early phase of a new stage. Overall, TSCAL which jointly uses the two knowledge transfer mechanisms achieves the best performance in terms of learning efficiency and stability. This result proves the superiority of the proposed TSCAL approach. We note the curves of TSCAL (LCO) and TSCAL (LOC) almost overlap in the later phase of the third learning stage, which means that there is no significant difference in the performance of the two variants of TSCAL. In other words, the final performance of TSCAL is independent of the order of subtasks.

2) Testing Results: We then tested the collision-avoiding flocking policy learned by TSCAL with ten followers in a new scenario. In the testing scenario, 20 obstacles were distributed in an area of 1100 m $\times$ 600 m (see Fig. 8). The leader flew along the predefined path without considering the surrounding obstacles. Our purpose was to test whether the followers would collide with the obstacles when the leader crossed them. This testing lasted for 180-time steps.

The trajectories of the UAV flock are visualized in Fig. 8. The performance of the learned TSCAL policy in this testing is shown in Fig. 9. Intuitively, the followers can successfully keep up with the leader in the cluttered environment, even

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

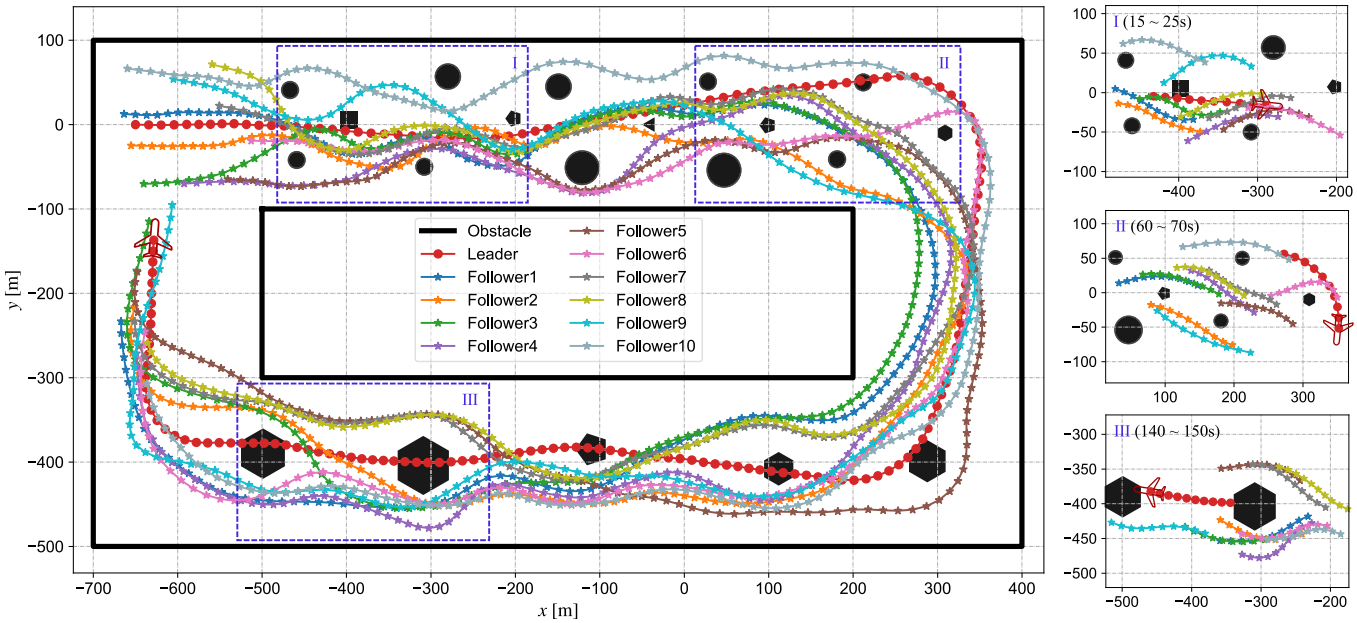


Fig. 8. Trajectories of the ten followers using the learned TSCAL policy to flock with the leader. The leader flies alone on the predefined path without considering the environmental obstacles. Note that the final position of the leader in its trajectory is marked with a UAV logo. A video about this simulation is available at https://youtu.be/R9yLJNYRIqY.

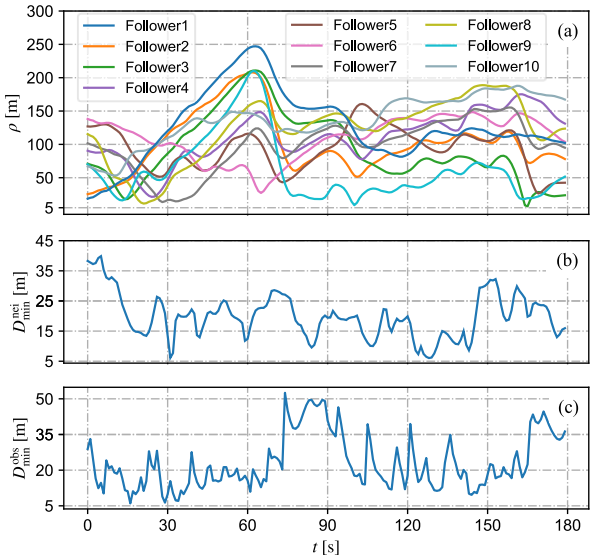


Fig. 9. Performance of the learned TSCAL policy. (a) Distances between the leader and the followers. (b) Minimum distance among the followers. (c) Minimum distance between the followers and surrounding obstacles.

when the leader conducts intense maneuvers. We can confirm this conclusion from Fig. 9(a). As shown, the distances between the leader and the followers are maintained in the range of 5–250 m. Most of the time, the fluctuation of the distance curves is not violent. This phenomenon means that the followers tend to maintain the current flight state if the surrounding situation does not change drastically. We note that the minimum value of $D_{\min}^{\text{nei}}$ is large than the minimum safety distance $R_s = 5$ m, which indicates that the followers can avoid colliding with each other. More importantly, the minimum value of $D_{\min}^{\text{obs}}$ is also large than R_s. In other words, there are no collisions between the follower and the surrounding obstacles throughout this testing. The above-mentioned results demonstrate that the followers using the learned TSCAL policy can achieve collision-free flocking in an unseen environment cluttered with various obstacles of different locations, shapes, and sizes, even if the leader's behavior is unreasonable and dangerous.

3) Generalization Results: We further tested the learned TSCAL policy with different numbers of followers and obstacles to quantitatively evaluate the generalization performance. Specifically, ten followers used the learned TSCAL policy to flock with the leader in the cluttered environment with 10–50 obstacles. We evaluated five random seeds for each scenario. Each evaluation consisted of 20 episodes, and each episode lasted for 180-time steps. Then, we expanded the number of followers to 15 and repeated the above-mentioned steps.

Fig. 10 and Table III present the generalization performance of the learned TSCAL policy by averaging the testing results, where T_{inf} represents the average inference time of each decision-making step. As shown, with the increase in the number of obstacles or followers, both the average distance and collision rate gradually increase, leading to a gradual decrease in the average reward. The clear reason is that the increase in the number of obstacles or followers inevitably results in a higher risk of collision. We note that even if in a narrow corridor-like space cluttered with 50 obstacles, the 15 followers employing the TSCAL policy are still able to maintain a distance to the leader of around 90 m and keep the collision rate below 4%. In addition, although the average inference time increases with the number of entities, TASCL only consumes less than 1.5 ms to make each decision for each follower. The above-mentioned results indicate that the learned TSCAL policy is able to achieve real-time flocking

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

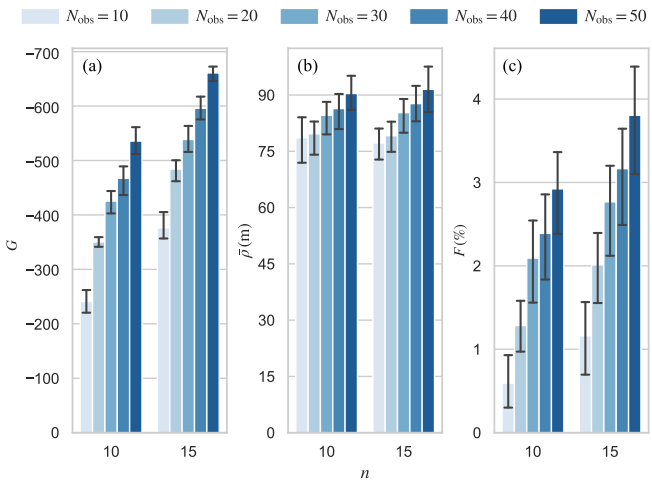


Fig. 10. Bar charts of the generalization performance of the learned TSCAL policy in cluttered environments with different numbers of obstacles. Note that the higher the bar, the worse the performance. (a) Average reward (note that the vertical axis is inversed). (b) Average distance. (c) Collision rate.

TABLE III
GENERALIZATION RESULTS OF THE LEARNED TSCAL POLICY IN CLUTTERED ENVIRONMENTS WITH DIFFERENT NUMBERS OF FOLLOWERS AND OBSTACLES

n	N_{obs}	G	$\bar{\rho}$ (m)	F (%)	T_{inf} (ms)
10	10	-240.99 ± 24.12	78.62 ± 6.92	0.59 ± 0.36	0.95
	20	-350.32 ± 10.45	79.68 ± 5.46	1.29 ± 0.36	1.04
	30	-425.47 ± 24.67	84.61 ± 5.07	2.09 ± 0.57	1.12
	40	-467.26 ± 30.77	86.40 ± 5.42	2.39 ± 0.62	1.11
	50	-535.63 ± 27.46	90.41 ± 5.38	2.92 ± 0.58	1.39
15	10	-376.40 ± 28.65	77.19 ± 5.00	1.16 ± 0.50	1.11
	20	-484.15 ± 22.37	79.17 ± 4.67	2.01 ± 0.48	1.21
	30	-538.66 ± 28.09	85.32 ± 5.19	2.77 ± 0.64	1.15
	40	-596.00 ± 23.92	87.72 ± 5.59	3.17 ± 0.69	1.22
	50	-660.89 ± 16.61	91.52 ± 6.91	3.81 ± 0.79	1.40

control and can be generalized to new environments cluttered with different numbers of obstacles and followers.

D. HITL Simulation Results

We finally evaluated the flocking policy learned with the proposed TSCAL approach in a high-fidelity HITL simulation platform.

1) Simulation Settings: We extended our high-fidelity HITL simulation platform developed in [32], [33] for flocking with multiple fixed-wing UAVs under a cluttered environment. In this platform, each UAV module consisted of a flight simulator, an autopilot, an onboard computer, and a ground control station. All UAV modules were accessed to a local area network for data sharing and communication.

The learned TSCAL policy was running independently on the onboard computer of each UAV with the robot operating system (ROS).[1] As shown in Fig. 11, we used the following ROS nodes to implement the flocking control: *communicator* node for receiving the neighbor's state via UDP socket, *detector* node for perceiving the surrounding obstacles, *MAVROS* node for communicating with Pixhawk[2] via MavLink[3] protocol, *commander* node for high-level decision-making based on the learned TSCAL policy, and *controller* node for low-level closed-loop control with a PID controller.

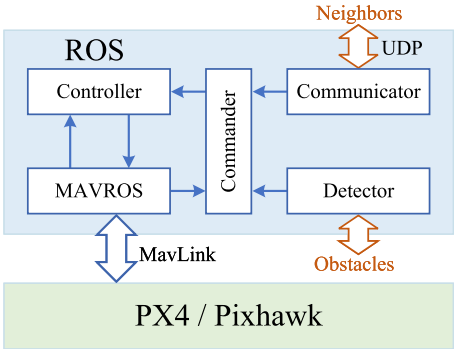


Fig. 11. Software architecture of the HITL platform.

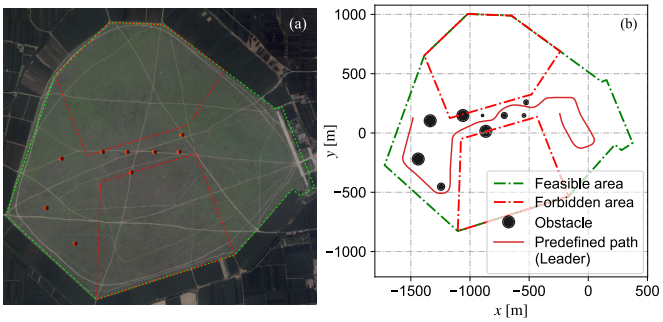


Fig. 12. Illustration of the task area for the HITL simulation. (a) Screenshot captured from the ground control station. (b) Predefined path of the leader.

In this HITL simulation, six followers (UAV24 ∼ 29) using the learned TSCAL policy to flock with the leader (UAV23) in an area with nine circular obstacles, as shown in Fig. 12(a). The leader's path was predefined as in Fig. 12(b). This simulation lasted for 240-time steps, i.e., 4 min. At each time step, the follower received the other UAVs' states from the communicator node and obtained its own state from the MAVROS node. The detector node was activated to perceive the locations and sizes of surrounding obstacles. According to the geometric relationship between the UAV and the obstacles, the output of LiDAR (i.e., the distances in nine directions) was easily calculated. Then, the set of shill agents was computed by (3). Based on the above-mentioned information, the commander node selected the roll rate and acceleration with the learned TSCAL policy. Finally, the roll angle and speed setpoints were determined and then published to the controller node for closed-loop control with a PID controller.

2) Simulation Results: The trajectories of the UAV flock in the HITL simulation are depicted in Fig. 13. As shown, the six followers are able to steadily keep up with the leader while avoiding collisions with surrounding obstacles. We note that the density of obstacles varies in different regions. Whether in a narrow corridor with obstacles [Fig. 13(a)] or in free space without obstacles [Fig. 13(b)], the followers are still able

[1] http://www.ros.org

[2] http://pixhawk.org

[3] https://mavlink.io

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

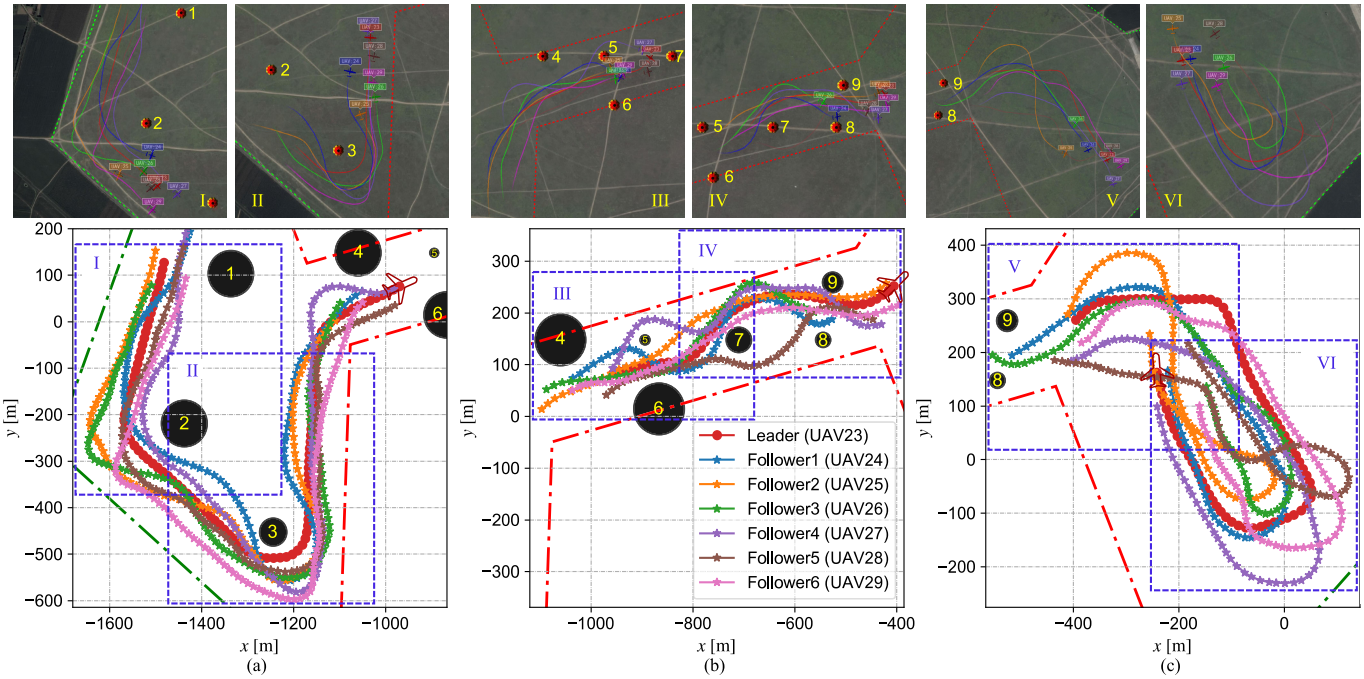


Fig. 13. Trajectories of the six followers flock with the leader using the learned TSCAL policy in the HITL simulation. Note that the six representative snapshots displayed on the top are captured from the ground control station. (a) $t = 0 \sim 110$ s. (b) $t = 110 \sim 155$ s. (c) $t = 155 \sim 240$ s. A video about this HITL simulation is available at https://youtu.be/R9yLJNYRIqY.

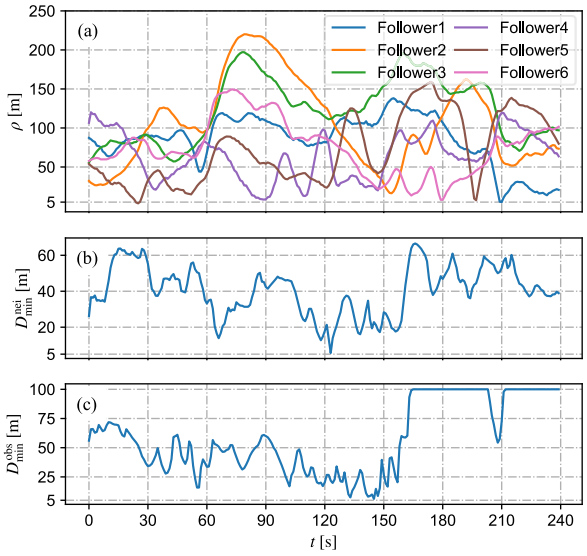


Fig. 14. Performance of the learned TSCAL policy in HITL simulation. (a) Distances between the leader and followers. (b) Minimum distance among followers. (c) Minimum distance between followers and surrounding obstacles.

to follow the leader without collision. This result shows that the flocking control policy learned with the proposed TSCAL approach enables the followers to adapt to different situations.

To better analyze the results of this HITL simulation, we also recorded the distances between the leader and the followers (ρ), the minimum distance among the followers ($D_{\min}^{\text{nei}}$), as well as the minimum distance between the followers and surrounding obstacles ($D_{\min}^{\text{obs}}$) in Fig. 14(a)–(c) with respect to time, respectively. It can be found that the average distance to the leader is kept between 5 and 150 m most of the time. This result confirms that the followers are able to follow the leader with relatively good performance. We note that the minimum value of $D_{\min}^{\text{nei}}$ [Fig. 14(b)] and the minimum value of $D_{\min}^{\text{obs}}$ [Fig. 14(c)] are both large than the minimum safety distance. In other words, using the learned TSCAL policy, the followers can successfully avoid collisions with each other and the surrounding obstacles.

VI. Conclusion

In this article, we propose a novel sample-efficient MADRL approach called TSCAL to solve the flocking and obstacle avoidance problem for multiple fixed-wing UAVs in cluttered environments. In TSCAL, a novel multiagent actor–critic DRL algorithm (i.e., HRAMA) and two knowledge transfer mechanisms (i.e., model reload and buffer reuse) are developed for online learning and offline transfer, respectively. Simulation results have demonstrated that TSCAL has significant advantages in policy optimality, sample efficiency, and learning stability. Besides, the model reload mechanism is more efficient than the buffer reuse mechanism. Using the learned optimal policy, a squad of fixed-wing UAVs is able to achieve collision-avoiding flocking in a complex environment cluttered with various obstacles of different shapes and sizes.

For future work, a meaningful direction is the development of automatic curriculum learning algorithms for efficient policy learning. Another promising direction is to ensure the safety of the learned policy through safe reinforcement learning. In addition, we will further develop the MADRL-based approach for flocking with large-scale UAV swarms in 3-D dynamic environments.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Cavagna et al., "Scale-free correlations in starling flocks," *Proc. Nat. Acad. Sci. USA*, vol. 107, no. 26, pp. 11865–11870, 2010.

[2] T. Vicsek and A. Zafeiris, "Collective motion," *Phys. Rep.*, vol. 517, no. 3, pp. 71–140, 2012.

[3] D. J. Evangelista, D. D. Ray, S. K. Raja, and T. L. Hedrick, "Three-dimensional trajectories and network analyses of group behaviour within chimney swift flocks during approaches to the roost," *Proc. Roy. Soc. B, Biol. Sci.*, vol. 284, no. 1849, Feb. 2017, Art. no. 20162602.

[4] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," *ACM SIGGRAPH Comput. Graph.*, vol. 21, no. 4, pp. 25–34, Jul. 1987.

[5] L. E. Beaver and A. A. Malikopoulos, "An overview on optimal flocking," *Annu. Rev. Control*, vol. 51, pp. 88–99, Jan. 2021.

[6] I. D. Couzin, J. Krause, R. James, G. D. Ruxton, and N. R. Franks, "Collective memory and spatial sorting in animal groups," *J. Theor. Biol.*, vol. 218, no. 1, pp. 1–11, Sep. 2002.

[7] R. Olfati-Saber, "Flocking for multi-agent dynamic systems: Algorithms and theory," *IEEE Trans. Autom. Control*, vol. 51, no. 3, pp. 401–420, Mar. 2006.

[8] S. Ha and H. Jeong, "Unraveling hidden interactions in complex systems with deep learning," *Sci. Rep.*, vol. 11, no. 1, p. 12804, Jun. 2021.

[9] G. Vásárhelyi, C. Virágh, G. Somorjai, T. Nepusz, A. E. Eiben, and T. Vicsek, "Optimized flocking of autonomous drones in confined environments," *Sci. Robot.*, vol. 3, no. 20, Jul. 2018, Art. no. eaat3536.

[10] Y. Lyu, J. Hu, B. M. Chen, C. Zhao, and Q. Pan, "Multivehicle flocking with collision avoidance via distributed model predictive control," *IEEE Trans. Cybern.*, vol. 51, no. 5, pp. 2651–2662, May 2021.

[11] F. Wang, J. Huang, K. H. Low, Z. Nie, and T. Hu, "AGDS: Adaptive goal-directed strategy for swarm drones flying through unknown environments," *Complex Intell. Syst.*, early access, Nov. 5, 2022, doi: 10.1007/s40747-022-00900-9.

[12] K. Klausen, T. I. Fossen, and T. A. Johansen, "Autonomous recovery of a fixed-wing UAV using a net suspended by two multirotor UAVs," *J. Field Robot.*, vol. 35, no. 5, pp. 717–731, Aug. 2018.

[13] Z. Liu et al., "Mission-oriented miniature fixed-wing UAV swarms: A multilayered and distributed architecture," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 3, pp. 1588–1602, Mar. 2022.

[14] H. Chen, Y. Cong, X. Wang, X. Xu, and L. Shen, "Coordinated path-following control of fixed-wing unmanned aerial vehicles," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 4, pp. 2540–2554, Apr. 2022.

[15] F. Schilling, F. Schiano, and D. Floreano, "Vision-based drone flocking in outdoor environments," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 2954–2961, Apr. 2021.

[16] P. Abichandani, C. Speck, D. Bucci, W. Mcintyre, and D. Lobo, "Implementation of decentralized reinforcement learning-based multi-quadrotor flocking," *IEEE Access*, vol. 9, pp. 132491–132507, 2021.

[17] E. Tolstaya, F. Gama, J. Paulos, G. Pappas, V. Kumar, and A. Ribeiro, "Learning decentralized controllers for robot swarms with graph neural networks," in *Proc. Conf. Robot Learn.*, 2020, pp. 671–682.

[18] C. Yan, X. Xiang, and C. Wang, "Towards real-time path planning through deep reinforcement learning for a UAV in dynamic environments," *J. Intell. Robotic Syst.*, vol. 98, pp. 297–309, Sep. 2020.

[19] C. Wang, J. Wang, Y. Shen, and X. Zhang, "Autonomous navigation of UAVs in large-scale complex environments: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2124–2136, Mar. 2019.

[20] Z. Sui, Z. Pu, J. Yi, and S. Wu, "Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 6, pp. 2358–2372, Jun. 2021.

[21] J. Wu, Z. Huang, W. Huang, and C. Lv, "Prioritized experience-based reinforcement learning with human guidance for autonomous driving," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 10, 2022, doi: 10.1109/TNNLS.2022.3177685.

[22] Z. Huang, J. Wu, and C. Lv, "Efficient deep reinforcement learning with imitative expert priors for autonomous driving," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jan. 26, 2022, doi: 10.1109/TNNLS.2022.3142822.

[23] G. Shen et al., "Deep reinforcement learning for flocking motion of multi-UAV systems: Learn from a digital twin," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11141–11153, Jul. 2022.

[24] A. Khan, E. Tolstaya, A. Ribeiro, and V. Kumar, "Graph policy gradients for large scale robot control," in *Proc. Conf. Robot Learn.*, 2020, pp. 823–834.

[25] S. Batra, Z. Huang, A. Petrenko, T. Kumar, A. Molchanov, and G. S. Sukhatme, "Decentralized control of quadrotor swarms with end-to-end deep reinforcement learning," in *Proc. 5th Conf. Robot Learn.*, 2022, pp. 576–586.

[26] X. Wang, Y. Chen, and W. Zhu, "A survey on curriculum learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 4555–4576, Sep. 2022.

[27] C. de Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulic, "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4552–4559, Jul. 2021.

[28] C. Xiao, P. Lu, and Q. He, "Flying through a narrow gap using end-to-end deep reinforcement learning augmented with curriculum learning and Sim2Real," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Sep. 6, 2021, doi: 10.1109/TNNLS.2021.3107742.

[29] L. Anzalone, P. Barra, S. Barra, A. Castiglione, and M. Nappi, "An end-to-end curriculum learning approach for autonomous driving scenarios," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 19817–19826, Oct. 2022.

[30] S.-M. Hung and S. N. Givigi, "A Q-learning approach to flocking with UAVs in a stochastic environment," *IEEE Trans. Cybern.*, vol. 47, no. 1, pp. 186–197, Jan. 2017.

[31] C. Yan, X. Xiang, and C. Wang, "Fixed-wing UAVs flocking in continuous spaces: A deep reinforcement learning approach," *Robot. Auto. Syst.*, vol. 131, Sep. 2020, Art. no. 103594.

[32] C. Yan, C. Wang, X. Xiang, Z. Lan, and Y. Jiang, "Deep reinforcement learning of collision-free flocking policies for multiple fixed-wing UAVs using local situation maps," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 1260–1270, Feb. 2022.

[33] C. Yan et al., "PASCAL: PopulAtion-specific curriculum-based MADRL for collision-free flocking with large-scale fixed-wing UAV swarms," *Aerosp. Sci. Technol.*, vol. 133, Feb. 2023, Art. no. 108091.

[34] A. H. Khan, S. Li, and X. Luo, "Obstacle avoidance and tracking control of redundant robotic manipulator: An RNN-based Metaheuristic approach," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4670–4680, Jul. 2020.

[35] X. Li, Z. Xu, S. Li, Z. Su, and X. Zhou, "Simultaneous obstacle avoidance and target tracking of multiple wheeled mobile robots with certified safety," *IEEE Trans. Cybern.*, vol. 52, no. 11, pp. 11859–11873, Nov. 2022.

[36] Z. Xu, X. Zhou, H. Wu, X. Li, and S. Li, "Motion planning of manipulators for simultaneous obstacle avoidance and target tracking: An RNN approach with guaranteed performance," *IEEE Trans. Ind. Electron.*, vol. 69, no. 4, pp. 3887–3897, Apr. 2022.

[37] C. Wang, J. Wang, J. Wang, and X. Zhang, "Deep-reinforcement-learning-based autonomous UAV navigation with sparse rewards," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6180–6190, Jul. 2020.

[38] Y. Jin, S. Wei, J. Yuan, and X. Zhang, "Hierarchical and stable multiagent reinforcement learning for cooperative navigation control," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 1, pp. 90–103, Jan. 2023.

[39] S. Zhang, Y. Li, and Q. Dong, "Autonomous navigation of UAV in multi-obstacle environments based on a deep reinforcement learning approach," *Appl. Soft Comput.*, vol. 115, Jan. 2022, Art. no. 108194.

[40] J. Wu, H. Wang, Y. Liu, M. Zhang, and T. Wu, "Learning-based fixed-wing UAV reactive maneuver control for obstacle avoidance," *Aerosp. Sci. Technol.*, vol. 126, Jul. 2022, Art. no. 107623.

[41] W. Wang et al., "From few to more: Large-scale dynamic multiagent curriculum learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 5, pp. 7293–7300.

[42] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–18.

[43] J. Chen et al., "Variational automatic curriculum learning for sparse-reward cooperative multi-agent problems," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 1–13.

[44] C. Bai, P. Yan, W. Pan, and J. Guo, "Learning-based multi-robot formation control with obstacle avoidance," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11811–11822, Aug. 2022.

[45] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized Pomdps*. New York, NY, USA: Springer 2016.

[46] T. Balch and R. C. Arkin, "Behavior-based formation control for multi-robot teams," *IEEE Trans. Robot. Autom.*, vol. 14, no. 6, pp. 926–939, Dec. 1998.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.

[47] J. Qi, J. Guo, M. Wang, C. Wu, and Z. Ma, "Formation tracking and obstacle avoidance for multiple quadrotors with static and dynamic obstacles," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 1713–1720, Apr. 2022.

[48] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6382–6393.

[49] H. Ryu, H. Shin, and J. Park, "Multi-agent actor-critic with hierarchical graph attention network," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 5, pp. 7236–7243.

[50] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Proc. Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)*, Sao Paulo, Brazil, 2017, pp. 66–83.

[51] R. E. Wang, M. Everett, and J. P. How, "R-MADDPG for partially observable environments and limited communication," 2020, *arXiv:2002.06684*.

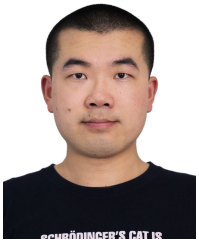
[52] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. NIPS Workshop Deep Learn.*, Dec. 2014, pp. 1–9.

[53] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[54] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 2961–2970.

Chao Yan received the B.E. degree in electrical engineering and automation from China University of Mining and Technology, Xuzhou, China, in 2017, and the M.S. degree in control science and engineering from the National University of Defense Technology, Changsha, China, in 2019, where he is currently pursuing the Ph.D. degree in control science and engineering with the College of Intelligence Science and Technology.

He is currently a Visiting Ph.D. Student with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore. His research interests include the applications of deep reinforcement learning techniques for the coordination control of unmanned aerial vehicle swarms.



Chang Wang received the B.S. degree in mathematics from the University of Science and Technology of China, Hefei, China, in 2003, the M.Sc. degree in applied mathematics from the National University of Defense Technology (NUDT), Changsha, China, in 2009, and the Ph.D. degree in robot learning from Delft University of Technology, Delft, The Netherlands, in 2016.

He is currently an Associate Professor with the College of Intelligence Science and Technology, NUDT. His research interests include reinforcement learning, developmental robotics, and human–robot teamwork.

Xiaojia Xiang received the B.E., M.S., and Ph.D. degrees in automatic control from the National University of Defense Technology (NUDT), Changsha, China, in 2003, 2007, and 2016, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, NUDT. His current research interests include mission planning and autonomous and cooperative control of unmanned systems.

Kin Huat Low received the B.S. degree in mechanical engineering from National Cheng Kung University, Tainan, Taiwan, China, in 1981, and the M.Sc. and Ph.D. degrees in mechanical engineering from the University of Waterloo, Waterloo, ON, Canada, in 1983 and 1986, respectively.

He is currently a Professor with the School of Mechanical and Aerospace Engineering, Nanyang Technological University (NTU), Singapore. His research interests include robotics, structural dynamics and vibrations, unmanned aerial vehicles, air traffic management, and urban air mobility.

Xiangke Wang (Senior Member, IEEE) received the B.E., M.E., and Ph.D. degrees in control science and engineering from the National University of Defense Technology (NUDT), Changsha, China, in 2004, 2006, and 2012, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, NUDT. He has authored or coauthored three books and more than 100 publications in peer-reviewed journals and international conferences. His current research interests include the control of multiagent systems and their applications on unmanned aerial vehicles.

Xin Xu (Senior Member, IEEE) received the B.S. degree in electrical engineering from the Department of Automatic Control, National University of Defense Technology (NUDT), Changsha, China, in 1996, and the Ph.D. degree in control science and engineering from the College of Mechatronics and Automation, NUDT, in 2002.

He has been a Visiting Scientist for cooperation research with The Hong Kong Polytechnic University, Hong Kong, the University of Alberta, Edmonton, AB, Canada, and the University of Strathclyde, Glasgow, U.K. He is currently a Full Professor with the College of Intelligence Science and Technology, NUDT, where he is also the Director of the Department of Artificial Intelligence. He received the Distinguished Young Scholars' Funds from the National Natural Science Foundation of China. He has authored or coauthored two monographs and more than 200 papers. His research interests include machine learning, autonomous control of robots, and intelligent unmanned systems.

Dr. Xu is one of the recipients of the second-class National Natural Science Award of China and the first-class Natural Science Awards of Hunan Province, China, twice. He is an Associate Editor of the IEEE TRANSACTIONS ON SYSTEM, MAN, AND CYBERNETICS: SYSTEMS, *Information Sciences*, the *International Journal of Robotics and Automation*, the Associate Editor-in-Chief of the *CAAI Transactions on Intelligence Technology*, and an Editorial Board Member of the *Journal of Control Theory and Applications*.

Lincheng Shen received the B.E., M.E., and Ph.D. degrees in automatic control from the National University of Defense Technology, Changsha, China, in 1986, 1989, and 1994, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, National University of Defense Technology, where he serves as the Dean of the Graduate School. His research interests include unmanned aerial vehicles, swarm robotics, and artificial intelligence.

Dr. Shen has been serving as an Editorial Board Member of the *Journal of Bionic Engineering* since 2007. He has initiated and organized several workshops and symposia, including the International Workshop on Bionic Engineering in 2012 and the Chinese Automation Congress in 2013.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:29:31 UTC from IEEE Xplore. Restrictions apply.